

closure_evidence_20260925

BOB-107 closure evidence — pre-dispatch subagent-input existence check

Date: 2026-09-25 **Item:** BOB-107 — §11.4.238 followup: pre-dispatch existence check for subagent task-brief source inputs

Summary

BOB-107 requires a pre-dispatch precondition helper: conductor-side tooling run BEFORE a downstream/consumer subagent is dispatched, that verifies the required-input files its task brief names genuinely exist on disk AND are non-empty. It must fail closed (non-zero exit, actionable “missing input, respawn the producer” message) rather than silently letting the downstream subagent discover the gap at task start (the original BOB-107 incident) or fabricate content unsupported by its named sources.

Files created

1. `scripts/check_subagent_inputs_exist.sh` — the pre-dispatch precondition helper itself. Executable bash script with a full §11.4.18-style documentation header (Purpose/Usage/Inputs/Outputs/Side-effects/ Dependencies/Cross-references).
2. `tests/unit/test_check_subagent_inputs_exist.sh` — the paired, real, executable test. Drives the real script (not a mock) against real files in a throwaway `mktemp -d` scratch directory.

No other files were touched. `docs/workable_items.db` was left untouched (its modified state predates this task and belongs to concurrent work); the `workable-items` CLI was not invoked; no `git add/commit/push` was run.

Design decision: how the helper is told which paths to check

Decision: explicit CLI arguments (one path per positional `PATH` arg), with an additional `--manifest FILE` option that reads a simple one-path-per-line text file (blank lines and `#`-prefixed comment lines ignored). Both forms can be combined; the underlying check logic is identical either way.

Why, not a brief-parsing/manifest-schema system:

I searched this project for an existing convention a subagent task brief uses to formally declare its “required input” paths:

- `.superpowers/sdd/` contains exactly one subdirectory (`qbittorrent-login-repair-verification/`) holding a single `progress.md` — a free-form prose ledger, not a structured manifest with a declared schema for required inputs.
- The evidence citation in the BOB-107 item text itself (`.superpowers/sdd/task-phase1a-report.md` line 21) no longer exists in the live tree (it was from a prior, now-cleaned session) — confirming there is no standing, versioned brief-manifest artifact this project maintains today.
- No task-brief generation script, YAML/JSON brief spec, or manifest schema exists anywhere under `scripts/`, `docs/`, or `.superpowers/` in this repository.

Given that, inventing a brief-parsing system would be building a mechanism that doesn’t correspond to anything this project actually uses — exactly the over-engineering the task description warned against. The honest, minimal design is: the CONDUCTOR (the one who read the brief and knows which paths it names as required) tells the helper explicitly, either as `argv` or as a plain manifest file. This is also directly analogous to this project’s own existing pattern for “helper takes explicit file paths” tooling (e.g. `scripts/generate_markdown_exports.sh FILE.md [...]`), so it follows an established local convention rather than a novel one.

If/when this project adopts a formal subagent task-brief manifest format in the future, `--manifest` gives a forward-compatible extension point: a brief-generation script could emit the required-inputs list as a one-path-per-line file and pipe it straight into `--manifest`, with zero changes to this helper's core logic.

Fail-closed + actionable message

- Exit 0: every declared path present and non-empty.
- Exit 1: at least one declared path missing or empty — prints an explicit CHECK-SUBAGENT-INPUTS: FAIL banner, names EXACTLY which path(s) failed and why ((missing) vs (empty, 0 bytes)), and prints an actionable remediation line: *“DO NOT dispatch the downstream/ consumer subagent on this brief. Remediation: respawn the producer subagent that was supposed to write the missing/empty file(s) above ... (§11.4.147(e))”*.
- Exit 2: usage error (no paths declared at all, or an unreadable `--manifest` file) — deliberately distinct from both 0 and 1, because a check that never ran asserted nothing; reporting that as a pass would be the §11.4.201(6) blind-instrument bluff this script exists to avoid.

Existence + non-emptiness check (why it matters for THIS item)

The script does not merely test `-e` a path. For each declared path it:

1. Confirms the path exists (`[[ -e "$p" ]]`), reporting MISSING otherwise.
2. Confirms it is a regular file, reporting a distinct message if it exists but is e.g. a directory.
3. Reads its byte size via a portable `file_size_bytes()` helper (tries GNU `stat -c%s`, then BSD/macOS `stat -f%z`, then falls back to `wc -c`) and reports EMPTY for any 0-byte file.

This directly targets the BOB-107 scenario: a producer subagent that crashed mid-write from an API rate limit is exactly the kind of failure that can leave a 0-byte or partially-written file on disk — a file that `test -e` alone would report as “present” while it is in fact useless, fabricatable-content-inducing evidence for the downstream consumer.

Anti-bluff: real, executable test — full output

The test constructs a real scratch directory (`mktemp -d`) with a mix of: present+non-empty files, an absent file, and a genuinely 0-byte file (the producer-crash scenario), then runs the real script against that mix through four cases: (1) mixed inputs via explicit args → must fail closed naming exactly the missing/empty files; (2) all inputs present → must exit 0; (3) the same mixed scenario via `--manifest` → must fail closed identically; (4) zero paths declared → must exit 2 (distinct from both PASS and FAIL).

Full test output (test run in this session, `bash tests/unit/test_check_subagent_inputs_exist.sh`):

```
=== CASE 1: mixed inputs (explicit args) -> must FAIL closed ===
    OK:      /tmp/bob107_check_inputs.xSNVvt/
curriculum_amendment_plan_v1.md (29 bytes)
    MISSING: /tmp/bob107_check_inputs.xSNVvt/
curriculum_analysis_modules_gap1.md
    EMPTY:   /tmp/bob107_check_inputs.xSNVvt/
curriculum_analysis_modules_gap2.md (0 bytes)
    OK:      /tmp/bob107_check_inputs.xSNVvt/
ai_curriculum_modules_27_35_extracted.md (29 bytes)

    CHECK-SUBAGENT-INPUTS: checked 4, failed 2

    CHECK-SUBAGENT-INPUTS: FAIL
    The following declared required-input file(s) are missing or
empty:
    - /tmp/bob107_check_inputs.xSNVvt/
curriculum_analysis_modules_gap1.md (missing)
    - /tmp/bob107_check_inputs.xSNVvt/
curriculum_analysis_modules_gap2.md (empty, 0 bytes)

    DO NOT dispatch the downstream/consumer subagent on this
brief.
    Remediation: respawn the producer subagent that was supposed
to write
    the missing/empty file(s) above, verify it completes without
hitting
    an API rate limit or other crash (§11.4.147(e)), then re-run
this
    check before dispatching the consumer subagent.
PASS: exits non-zero on missing+empty inputs (rc=1)
PASS: output names the exact missing path
PASS: output names the exact empty path
PASS: output classifies the missing file as MISSING
PASS: output classifies the 0-byte file as EMPTY
```

PASS: output gives the actionable respawn-the-producer remediation

PASS: present non-empty file (/tmp/bob107_check_inputs.xSNVvt/curriculum_amendment_plan_v1.md) reported OK, not a false failure

=== CASE 2: all inputs present + non-empty -> must exit 0 ===

OK: /tmp/bob107_check_inputs.xSNVvt/

curriculum_amendment_plan_v1.md (29 bytes)

OK: /tmp/bob107_check_inputs.xSNVvt/

ai_curriculum_modules_27_35_extracted.md (29 bytes)

CHECK-SUBAGENT-INPUTS: checked 2, failed 0

CHECK-SUBAGENT-INPUTS: OK – all declared inputs present and non-empty.

PASS: exits 0 when all declared inputs present and non-empty (rc=0)

PASS: success output reports OK

=== CASE 3: mixed inputs via --manifest -> must FAIL closed ===

OK: /tmp/bob107_check_inputs.xSNVvt/

curriculum_amendment_plan_v1.md (29 bytes)

MISSING: /tmp/bob107_check_inputs.xSNVvt/
curriculum_analysis_modules_gap1.md

EMPTY: /tmp/bob107_check_inputs.xSNVvt/
curriculum_analysis_modules_gap2.md (0 bytes)

CHECK-SUBAGENT-INPUTS: checked 3, failed 2

CHECK-SUBAGENT-INPUTS: FAIL

The following declared required-input file(s) are missing or empty:

- /tmp/bob107_check_inputs.xSNVvt/
curriculum_analysis_modules_gap1.md (missing)

- /tmp/bob107_check_inputs.xSNVvt/
curriculum_analysis_modules_gap2.md (empty, 0 bytes)

DO NOT dispatch the downstream/consumer subagent on this brief.

Remediation: respawn the producer subagent that was supposed to write

the missing/empty file(s) above, verify it completes without hitting

an API rate limit or other crash (\$11.4.147(e)), then re-run this

check before dispatching the consumer subagent.

PASS: --manifest form exits non-zero on missing+empty inputs (rc=1)

PASS: --manifest form output names the exact missing path

=== CASE 4: no paths declared -> must exit 2 (cannot-run, not a pass) ===

ERROR: no input paths declared (pass PATH args and/or --

```
manifest FILE with at least one)
  Usage: check_subagent_inputs_exist.sh PATH [PATH ...]
        check_subagent_inputs_exist.sh --manifest MANIFEST_FILE
[PATH ...]
```

Verifies each declared subagent task-brief required-input path exists AND

is non-empty. Run this BEFORE dispatching a downstream/ consumer subagent

whose brief names required-read source files, so a producer crash (e.g. an

API-rate-limit crash mid-write, §11.4.147(e)) is caught BEFORE dispatch

instead of discovered by the downstream subagent at task start.

Exit 0 all declared paths present and non-empty.

Exit 1 at least one path is missing or empty -- DO NOT dispatch; respawn
the producer subagent that was supposed to write it.

Exit 2 usage error (no paths given, or the --manifest file itself is

unreadable) -- the check could not run.

CHECK-SUBAGENT-INPUTS: CANNOT-RUN (no declared paths)

PASS: no-args invocation exits 2 (usage error, distinct from PASS/FAIL)

RESULT: 12 passed, 0 failed
exit=0

12/12 assertions passed. bash -n syntax check clean on both the script and its test.

Assumptions not fully specified by the item text

1. **Exit-code scheme.** The item text does not specify exact exit codes. I followed the existing local convention set by scripts/ownership_precondition.sh (another conductor/orchestration fail-closed precondition script in this project): 0 = pass, 1 = the precondition genuinely failed (block dispatch), 2 = the check itself could not run (usage error) — distinct from both, per that script’s own documented rationale (“a check that could not run has asserted nothing”).
2. **Script location and name.** Placed under scripts/ (not scripts/pre_build/) per the item’s own guidance that this is meta/ orchestration tooling for the conductor’s workflow, not a pre-build

gate on the shipped product. Used the exact name suggested in the item text, `scripts/check_subagent_inputs_exist.sh`, since it already matches this project's lowercase-snake_case naming convention (§11.4.29) and the pattern of other single-purpose helper scripts (`ownership_precondition.sh`, `generate_markdown_exports.sh`).

3. **Test location.** Placed under `tests/unit/` (not `tests/pre_build/`) following the precedent of `scripts/ownership_precondition.sh` → `tests/unit/test_ownership_precondition.sh` and `scripts/generate_markdown_exports.sh` → `tests/unit/test_generate_markdown_exports_path_arg.sh` — every existing project-tooling (not shipped-product) bash script in this repo that has a paired test keeps that test under `tests/unit/`.
4. **“Regular file, not a directory” handling.** Not explicitly required by the item, but added as a distinct MISSING-class outcome (rather than crashing on a directory when reading its size) since a brief could plausibly name a directory by mistake and the helper should still fail closed with a clear reason rather than erroring opaquely.
5. **Manifest comment syntax (#-prefixed lines, blank lines skipped).** Not specified by the item; added as a minor convenience so a brief author can annotate WHY each path is required directly in the manifest file, without inventing any parsing beyond simple line-based skipping.

Non-compliance checks

- No `git add`, `git commit`, `git push`, or `scripts/commit-push-all.sh` invocation was performed.
 - `docs/workable_items.db` was not modified by this task (its pre-existing modified state in `git status` predates and is unrelated to this work).
 - The `workable-items` CLI was not invoked.
 - No files outside the declared scope (`scripts/check_subagent_inputs_exist.sh`, `tests/unit/test_check_subagent_inputs_exist.sh`, and this evidence file under `docs/qa/BOB-107/`) were touched.
-

CORRECTION (2026-09-25, same session — conductor-caught duplicate)

The subagent's dispatched investigation missed a pre-existing implementation. While independently verifying a LATER, unrelated subagent's work (BOB-077), the conductor discovered tests/hooks/test_check_brief_inputs.sh — a TRACKED test file, committed at 7b45113c6b03524d9c799bdd68496ec202575e4c (2026-08-21, the SAME commit that also closed BOB-114), testing scripts/hooks/check-brief-inputs.sh — a pre-dispatch precondition checker for subagent task-brief required inputs, already implemented, already self-tested, and ALREADY closing this item's exact acceptance criterion:

```
$ bash scripts/hooks/check-brief-inputs.sh --self-test
[check-brief-inputs] $11.4.107(10) self-test – golden-good /
golden-bad / empty-file control needle
    golden-good      PASS  (two present non-empty inputs -> exit 0)
    golden-bad       PASS  (missing input AND empty input both named;
exit 1; remediation printed)
    brief-extract    PASS  ('...' shorthand resolved correctly; out-
of-section path NOT scanned; missing sibling detected)
[check-brief-inputs] self-test PASS – oracle validated in both
polarities
```

This pre-existing script is MORE capable than the one this subagent built from scratch: beyond the same existence/non-emptiness checks, it can extract required-input paths directly from a brief file's own “...” shorthand notation, rather than requiring the conductor to pass an explicit path list — closer to the item's own stated ideal (“a small script the conductor runs before Task/Agent dispatch when a brief names required input paths”).

Root cause of the miss: the subagent's dispatch brief (authored by the conductor) restricted its WRITE scope to a new script + test and did not direct it to search scripts/hooks/ for a pre-existing solution before building one — an incomplete application of §11.4.74 catalogue-first discovery on the conductor's part, not a fabrication or bluff on the subagent's part (its own investigation of .superpowers/sdd/ for a brief-manifest CONVENTION was genuine and thorough; it simply never searched for an existing IMPLEMENTATION under a differently-scoped directory it had been told only to search, not write to).

Remediation applied: scripts/check_subagent_inputs_exist.sh and tests/unit/test_check_subagent_inputs_exist.sh (the subagent's newly built, now-confirmed-redundant pair) were REMOVED — safe, since neither was ever committed and grep -rl

check_subagent_inputs_exist across the tree found zero references outside the pair itself and this evidence file. This item's closure is CORRECT (the acceptance criterion genuinely IS met) but its evidence citation is now corrected to point at the real, pre-existing, better-designed implementation (scripts/hooks/check-brief-inputs.sh / tests/hooks/test_check_brief_inputs.sh, commit 7b45113) rather than a redundant fork this session almost shipped alongside it.